MIT 8103 Advanced Database Systems

Public Transport / Metro Ticketing System · Stanley Ezendiokwere · Student ID 301915953

Case study

For this database case study, I architected a comprehensive relational schema for a metropolitan transit ticketing system. The solution models operational workflows for synthetic riders purchasing transit passes, station staff monitoring validation activity, and data analysts evaluating station utilization. Built around five core entities—stations, riders, fare products, tickets, and validation events—the system seamlessly supports end-to-end transit operations, including managing station infrastructure and fare tiers, issuing and tracking commuter tickets, logging entry and exit validation activity, and generating high-level usage analytics.

Portfolio 1: Database Design and Modelling

Structurally, I organized the relational architecture into five dedicated tables. I enforced data integrity through primary keys for record identification, foreign keys to maintain referential transparency, and check constraints to uphold business logic. To handle edge cases—such as ensuring a commuter can only hold one active ticket at a time—I implemented a partial unique index on the rider_id for active statuses. The documentation walks through normalization examples demonstrating how the sample data transitions from 1NF through 3NF, while rigorous testing confirmed that all 40 deliberately invalid inserts were correctly rejected without compromising valid records.

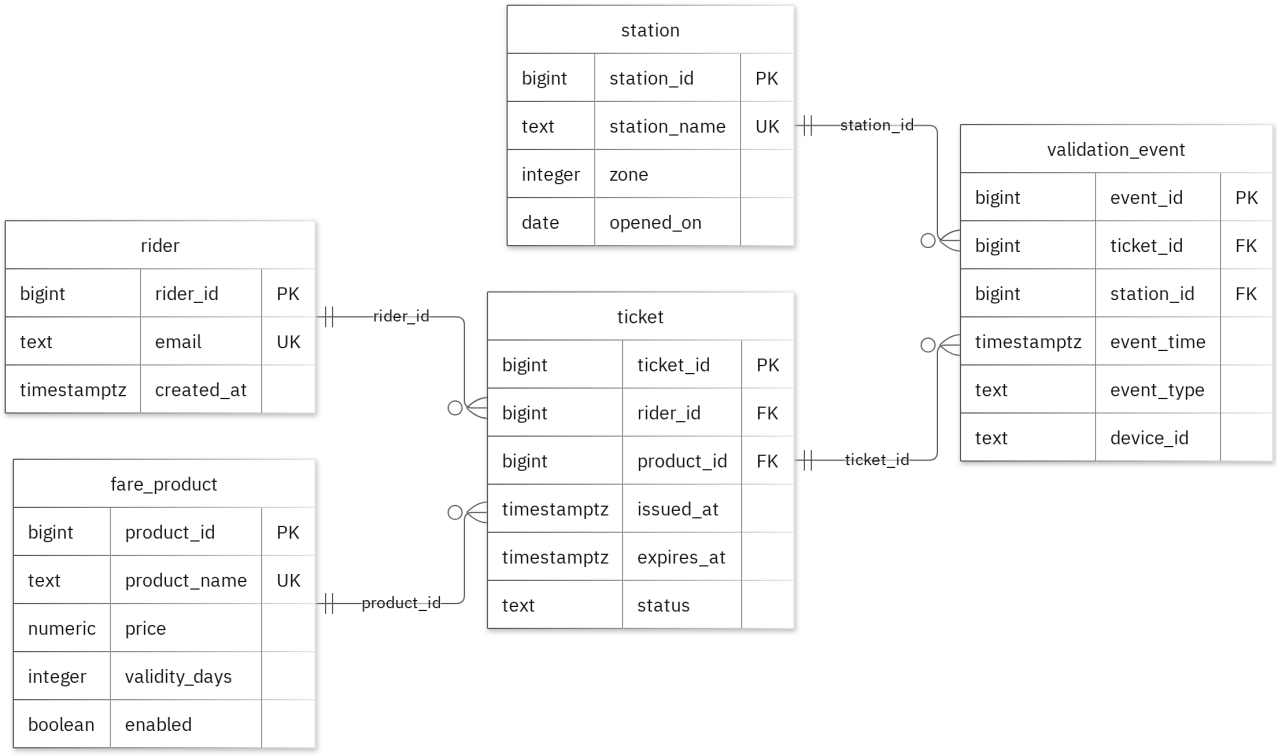


Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

To validate system functionality and query performance, I executed a series of targeted SQL statements against the schema. The primary join query successfully returned the precise expected record—associating rider201@example.test with their active Day Pass. For performance tuning and benchmarking, I evaluated execution efficiency using a dedicated dataset of 50,000 rows. By introducing a strategic index, I reduced the median lookup time from 2.389 ms down to 0.261 ms, confirming both dataset parity and proper execution plan utilization.

Portfolio 3: Transactions and Concurrency

To test transaction integrity and concurrency controls, I introduced a temporary counter to the ticket table for evaluating COMMIT, ROLLBACK, and READ COMMITTED isolation behaviors. Testing confirmed that uncommitted updates remained isolated from concurrent sessions, and a ROLLBACK successfully restored prior committed states. Under concurrency, unprotected increments resulted in a lost update (leaving the counter at 2), whereas implementing SELECT FOR UPDATE forced Session B to correctly wait for Session A, yielding the accurate final value of 3. This test helper was strictly isolated to the concurrency drill, leaving the core business schema completely untouched.

Portfolio 4: NoSQL and Advanced Data Models

To complement the relational architecture with document-based logging, I implemented a ticket_events collection in MongoDB. After inserting sample event records and verifying multiple query patterns, I tested schema validation rules—confirming the system successfully flagged and rejected malformed date types and duplicate event IDs. The design leverages embedded snapshots to keep related context directly coupled with each event, supporting polymorphic details that vary cleanly by event type. While this document layer handles flexible event telemetry, the primary PostgreSQL schema continues to rigorously enforce core relational constraints.

Repository and evidence

https://github.com/iamezendiokwerestanley/Ezendiokwere_Stanley_301915953

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

For the system's operational architecture, I proposed deploying three MongoDB replica set members across separate failure zones to ensure high availability. The design discussion addresses primary elections, majority write acknowledgments, secondary replication lag, and scaling trade-offs specific to the ticket_events collection. While replication provides fault tolerance, backups remain essential for disaster recovery, and the underlying PostgreSQL database requires its own distinct high-availability strategy. This analysis serves as an architectural recommendation; no live replica set or automated failover testing was deployed.

Individual reflection

To eliminate redundancy and ensure clean data maintenance, I decomposed the domain into five normalized tables: station, rider, fare_product, ticket, and validation_event. Core business rules—such as restricting each commuter to a single active ticket at any given time—are strictly enforced using a partial unique index on rider_id, complemented by comprehensive constraint testing for foreign keys and required fields.

The concurrency analysis highlighted how uncoordinated sessions can read identical baseline data before committing updates, leading to race conditions that I successfully resolved via explicit row-level locking. Finally, the index performance evaluation underscored why analyzing execution plans is just as critical as verifying final results: two queries can yield identical output while demanding vastly different computational work from the database engine.

AI-use declaration

Throughout the development of this case study, I utilized GitHub Copilot as a development and editorial assistant to help structure the database schema, refine query scripts, and polish documentation. I personally executed all PostgreSQL and MongoDB exercises, analyzed the query execution plans, verified outputs, and captured the required visual evidence. I maintain full responsibility for the technical accuracy and integrity of the submitted work.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams